

Protokoll Parallelisierung und Synchronisierung

Parallelisierung:

Für jede gesuchte Datei wird ein eigener Child-Prozess mit **fork()** erstellt. Der Parent-Prozess erzeugt all benötigten Child-Prozesse innerhalb einer Schleife, ohne auf deren Beendigung zu warten.

Jeder Child-Prozess führt unabhängig die funktion **searchFile()** für seinen eigenen Dateinamen aus. Dadurch können mehrere Suchvorgänge gleichzeitig ausgeführt werden.

Die Prozess-IDs der erzeugten Child-Prozesse werden im Parent gespeichert. Nachdem die Ergebnisse verarbeitet wurden, wartet der Parent mit **waitpid()** auf die Beendigung aller Child-Prozesse. Dadurch werden Zombie-Prozesse verhindert.

Synchronisierung der Ausgabe:

Damit mehrere Child-Prozesse nicht gleichzeitig direkt auf stdout schreiben und sich ihre Ausgaben vermischen, wird ein Unnamed Pipe verwendet.

Der Child-Prozess schließt zunächst den nicht benötigten Lese-Descriptor **pipefd[0]**. Nach der Suche schreibt er seine Ergebnisse mit **write()** in die Pipe und schließt den Schreib-Descriptor **pipefd[1]**.

Der Parent-Prozess schließt seinen nicht benötigten Schreib-Descriptor **pipefd[1]**. Er liest die Ergebnisse mit **read()** aus der Pipe und gibt sie auf stdout aus.

Damit schreibt nur der Parent-Prozess die Suchergebnisse auf stdout. Die Kommunikation zwischen Child- und Parent-Prozessen erfolgt über die Pipe.